

JavaScript DOM

(Document Object Model)

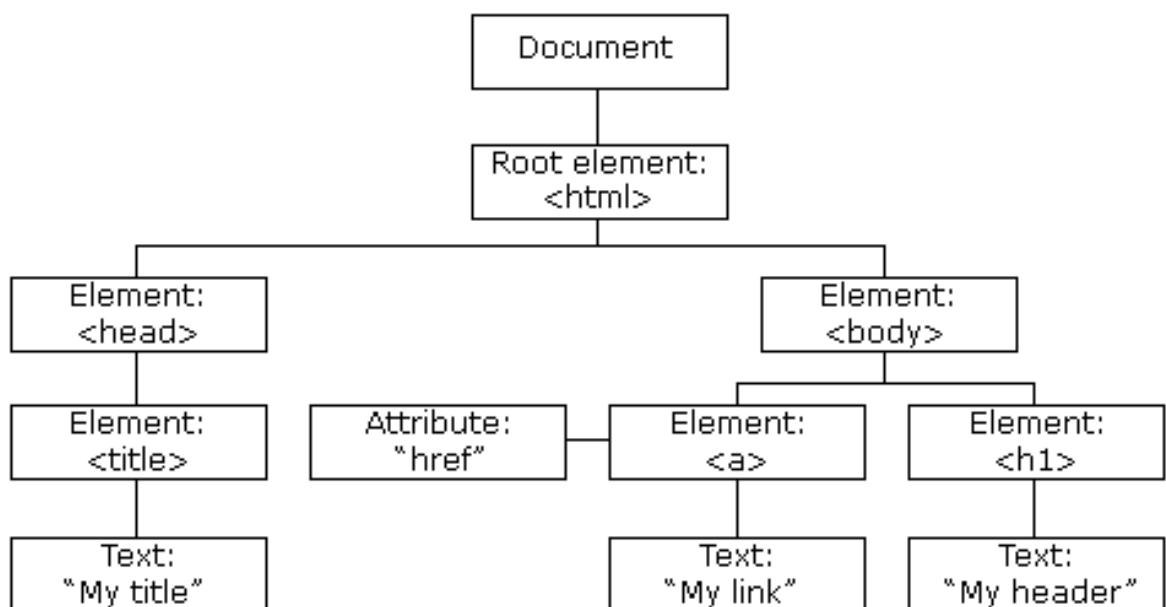
DOM (Document Object Model) is a way for JavaScript to interact with and change HTML and CSS.

Think of the DOM as a **family tree** of your webpage, where each element (like `<p>`, `<div>`, `<button>`, etc.) is a part of the tree.

With the HTML DOM, JavaScript can access and change all the elements of an HTML document.

When a web page is loaded, the browser creates a Document Object Model of the page.

The HTML DOM Tree of Objects



With the **DOM**, JavaScript has full control over a webpage, allowing it to:

- ✓ **Modify Content:** Change any HTML element on the page.
- ✓ **Modify Attributes:** Update attributes like `src`, `href`, `alt`, etc.
- ✓ **Modify Styles:** Dynamically change CSS styles.

- ✓ **Remove Elements:** Delete existing HTML elements and attributes.
- ✓ **Add Elements:** Create and insert new HTML elements and attributes.
- ✓ **Handle Events:** React to user actions like clicks, typing, or scrolling.
- ✓ **Create Events:** Generate new events like custom notifications or animations.

The **DOM (Document Object Model)** is a **W3C standard** that defines how web pages can be accessed and changed using JavaScript.

Types of DOM:

1. **Core DOM** – Works for all document types.
2. **XML DOM** – Works with XML documents.
3. **HTML DOM** – Works with HTML documents.

What is the HTML DOM?

The **HTML DOM** is a set of rules that allow JavaScript to **get, change, add, or remove** elements on a webpage, making it dynamic and interactive.

JavaScript - HTML DOM Methods

HTML DOM methods are actions you can perform (on HTML Elements).

HTML DOM properties are values (of HTML Elements) that you can set or change.

```
<p id="demo"></p>

<script>
    document.getElementById("demo").innerHTML = "Hello World!";
</script>
```

In the example above, `getElementById` is a method, while `innerHTML` is a property.

HTML DOM Document Object

If you want to access any element in an HTML page, you always start with accessing the document object.

Finding HTML Elements

document.getElementsByTagName(name)

```
<p>Hello 1</p>
<p>Hello 2</p>

<button onclick="changeText()">Click Me</button>

<script>
  function changeText() {
    document.getElementsByTagName("p")[1].innerHTML = "Text changed!";
  }
</script>
```

document.getElementsByClassName(name)

```
<p class="highlight">Text 1</p>
<p class="highlight">Text 2</p>

<button onclick="changeColor()">Click Me</button>

<script>
  function changeColor() {
    document.getElementsByClassName("highlight")[1].style.color = "red";
  }
</script>
```

document.querySelectorAll()

If you want to find all HTML elements that match a specified CSS selector (id, class names, types, attributes, values of attributes, etc), use the `querySelectorAll()` method.

```
<p class="intro">Hello World!</p>
```

```
<p class="intro">This example explains <b>querySelectorAll</b> method.</p>
```

```
<p id="demo"></p>
```

```
<script>
```

```
    const x = document.querySelectorAll("p.intro");
```

```
    document.getElementById("demo").innerHTML = x[1].innerHTML;
```

```
</script>
```

DOM Manipulation

tagName : returns tag for element nodes

innerText : returns the text content of the element and all its children

innerHTML : returns the plain text or HTML contents in the element

textContent : returns textual content even for hidden elements

getAttribute(attr) : to get the attribute value

setAttribute(attr, value) :to set the attribute value

className = " "

classList : add(),remove(),toggle()

node.style : adding style to the elements

node.append(el) : adds at the end of node (inside)

node.prepend(el) : adds at the start of node (inside)

node.before(el) : adds before the node (outside)

node.after(el) : adds after the node (outside)

node.remove() : removes the node

Practice

Qs. Create a H2 heading element with text - "Hello JavaScript" . Append "from Felix its system students" to this text using JS.

Qs. Create 3 divs with common class name - "box" .Access them & add some unique text to each of them

Qs. Create a new button element. Give it a text "click me" , background color of red & text color of white. Insert the button as the first element inside the body tag.

Qs. Create a <p> tag in html, give it a class & some styling.
Now create a new class in CSS and try to append this class to the <p> element.
Did you notice, how you overwrite the class name when you add a new one?
Solve this problem using classList.

How to Work with Forms and Inputs?

We can **get user input** and **display it**.

Example

```
<input type="text" id="nameInput" placeholder="Enter your name">
```

```
<button onclick="showName()">Submit</button>
```

```
<p id="output"></p>
```

```
<script>
```

```
function showName() {  
    let name = document.getElementById("nameInput").value;  
    document.getElementById("output").innerText = "Hello, " + name;  
}
```

```
</script>
```

Example

```
<body>
```

```
<h2>JavaScript Validation</h2>
```

```
<form name="myForm" onsubmit="return validateForm()" method="post">  
  Name: <input type="text" name="fname">  
  <input type="submit" value="Submit">  
</form>
```

```
<script>  
  function validateForm() {  
    let x = document.forms["myForm"]["fname"].value;  
    if (x == "") {  
      alert("Name must be filled out");  
      return false;  
    }  
  }  
</script>
```

```
</body>
```

HTML DOM Events

HTML DOM allows JavaScript to react to HTML events.

Events are actions like **clicking a button, typing in a field, or moving the mouse**.

Examples of HTML events:

- When a user clicks the mouse
- When a web page has loaded

- When an image has been loaded
- When the mouse moves over an element
- When an input field is changed
- When an HTML form is submitted

Common JavaScript Events

Event Name	When It Happens
<code>onclick</code>	When an element is clicked
<code>onmouseover</code>	When the mouse moves over an element
<code>onmouseout</code>	When the mouse moves away from an element
<code>onchange</code>	When the value of an input field changes
<code>onkeydown</code>	When a key is pressed on the keyboard
<code>onload</code>	When the webpage finishes loading

`onclick` / `ondblclick` - When an element is clicked

```
<button onclick="changeText()">Click Me</button>
<p id="demo">Hello!</p>

<script>
  function changeText() {
    document.getElementById("demo").innerHTML = "You clicked the
button!";
  }
</script>
```

`onmouseover` - When the mouse moves over an element

```
<div id="box" onmouseover="changeColor()" style="width:100px;
height:100px; background-color:lightblue;"></div>

<script>
  function changeColor() {
```

```
        document.getElementById("box").style.backgroundColor = "yellow";
    }
</script>
```

onmouseout - When the mouse moves away from an element

```
<div id="box" onmouseout="resetColor()" style="width:100px; height:100px;
background-color:lightblue;"></div>
```

```
<script>
    function resetColor() {
        document.getElementById("box").style.backgroundColor = "red";
    }
</script>
```

onchange - When an input value changes

```
<body>
```

```
<select onchange="showValue(this)">
    <option value="Apple">Apple</option>
    <option value="Banana">Banana</option>
    <option value="Cherry">Cherry</option>
</select>
<p id="output"></p>
```

```
<script>
    function showValue(selectBox) {
        document.getElementById("output").innerHTML = "You selected: " +
selectBox.value;
    }
</script>
```

```
</body>
```

onload - When the page loads

```
<body onload="showAlert()">
```

```
<script>
```

```
        function showAlert() {
            alert("Page has loaded!");
        }
    </script>
</body>
```

HTML DOM EventListener

What is HTML DOM `addEventListener()`?

The `addEventListener()` method allows you to **attach an event** to an element **without overwriting existing events**.

- ◆ Normally, if you use `onclick` multiple times on the same element, the **last one** replaces the previous one.
- ◆ But with `addEventListener()`, you can **add multiple event handlers** without replacing the old ones.

Syntax

```
element.addEventListener(event, function, useCapture);
```

Example

```
<body>

    <button id="myBtn">Try it</button>

    <script>
        document.getElementById("myBtn").addEventListener("click", function ()
    {
        alert("Hello World!");
    });
    </script>

</body>
```

Why Use `addEventListener()` Instead of `onclick`?

Problem with `onclick` (Overwriting Events)

```
<button id="btn">Click Me</button>

<script>
document.getElementById("btn").onclick = function() {
    alert("First Event");
};
document.getElementById("btn").onclick = function() {
    alert("Second Event"); // This replaces the first one
};
</script>
```

 **Only "Second Event" will run because the first event gets replaced.**

Solution: Using `addEventListener()`

👉 Multiple events **can run without overwriting** each other.

```
<button id="btn">Click Me</button>

<script>
document.getElementById("btn").addEventListener("click", function() {
    alert("First Event");
});
document.getElementById("btn").addEventListener("click", function() {
    alert("Second Event");
});
</script>
```

✅ Now, both "First Event" and "Second Event" will run when the button is clicked.

Let's Practice

Qs. Create a toggle button that changes the screen to dark-mode when clicked & light-mode when clicked again.